

LinkedInPost V2

Plateforme de Content Operations
et Social Publishing assiste par IA

Document d'architecture technique - Version 2.0

Stack : Next.js 16 + React 19 + Prisma + PostgreSQL

Mai 2026

Table des matieres

1. Vision V2	5
1.1 Objectifs strategiques	5
1.2 Principes directeurs	5
2. Architecture technique V2	6
2.1 Stack technique	6
2.2 Architecture en couches	6
2.3 Diagramme d'architecture globale	7
2.4 Gestion des environnements	7
3. Schema de base de donnees V2	8
3.1 Nouveaux modeles	8
3.1.1 Workspace (Espace de travail)	8
3.1.2 BrandProfile (Branding editorial)	9
3.1.3 PromptTemplate (Bibliotheque de prompts)	10
3.1.4 Media (Bibliotheque mediatique)	10
3.1.5 PostMedia (Association post-medias)	11
3.1.6 ContentScore (Scores de qualite)	11
3.1.7 CalendarEvent (Evenements calendrier)	12
3.1.8 SocialAccount (Comptes sociaux multi-reseaux)	13
3.1.9 ValidationStep (Workflow multi-approbateurs)	13
3.1.10 Notification	14
3.2 Modeles modifies (evolution V1)	15
3.3 Relations et diagramme ER	15
4. Nouveaux ecrans	16
4.1 Vue calendrier editorial	16
4.2 Bibliotheque de prompts	17
4.3 Branding editorial	17
4.4 Media Library	17

4.5 Dashboard analytics	18
4.6 Gestion des workspaces	18
5. Nouvelles API routes	18
5.1 APIs Calendrier	18
5.2 APIs Prompts	19
5.3 APIs Branding	19
5.4 APIs Medias	20
5.5 APIs Analytics	20
5.6 APIs Workspaces	20
5.7 APIs Réseaux sociaux (multi-réseaux)	21
5.8 APIs Validation avancée	21
5.9 APIs Notifications	21
6. Code et pseudo-code détaillé	22
6.1 Schema Prisma V2 - Extrait	22
6.2 Service de génération IA enrichie	23
6.3 Service de scoring de qualité	24
6.4 Service de publication multi-réseaux	25
6.5 Service de calendrier (drag and drop)	26
6.6 Middleware d'authentification	27
6.7 Scheduler de publications différées	28
7. Plan de migration V1 vers V2	29
7.1 Phase 1 - Fondations (semaines 1-3)	30
7.2 Phase 2 - Branding et Prompts (semaines 4-6)	30
7.3 Phase 3 - Calendrier et Medias (semaines 7-10)	30
7.4 Phase 4 - Analytics et Gouvernance (semaines 11-14)	31
7.5 Phase 5 - Multi-réseaux et Industrialisation (semaines 15-18)	31
7.6 Script de migration automatisé	32
8. Stratégie produit V1 vers V2	33
8.1 Positionnement	33

8.2 Roadmap produit	33
8.3 Metriques de succes	34
8.4 Model d'evolution et scalabilite	34
8.5 Risques et mitigation	35

1. Vision V2

La V1 de LinkedInPost a demontre la faisabilite d'un mini SaaS interne de gestion de posts LinkedIn avec generation IA, validation humaine et publication directe. La V2 vise a transformer cet outil en une plateforme complete de **Content Operations** et de **Social Publishing** assistee par IA, capable de gerer l'ensemble du cycle de vie editorial d'une organisation, du brainstorming initial a la publication multi-reseaux, en passant par la creation de contenu enrichi (carrousels, scripts video, visuels) et le suivi analytique des performances.

L'objectif strategique est de passer d'un outil de publication simple a un veritable hub centralise de gestion editoriale. Les equipes marketing et communication pourront planifier, creer, valider, programmer et publier du contenu sur plusieurs reseaux sociaux depuis une interface unique, tout en beneficiant de l'assistance IA pour la creation, l'optimisation et l'analyse des performances. La plateforme integrera des concepts de gouvernance avancee avec des workflows multi-approbateurs, une gestion multi-marques et multi-pages, ainsi qu'un systeme de monitoring et d'industrialisation robuste pour garantir la fiabilite en production.

Cette evolution s'inscrit dans une logique de produit scalable, ou chaque module est concu pour fonctionner de maniere autonome tout en s'integrant dans un ecosysteme coherent. L'architecture doit permettre l'ajout progressif de fonctionnalites sans remettre en cause l'existant, et la migration depuis la V1 doit se faire de maniere transparente pour les utilisateurs existants.

1.1 Objectifs strategiques

- Centraliser toute la gestion editorial dans une plateforme unique
- Automatiser la creation de contenu grace a une IA multi-fournisseur
- Industrialiser le processus de validation et de publication
- Permettre la gestion multi-marques et multi-pages LinkedIn
- Preparer l'extension multi-reseaux (X/Twitter, Facebook, Instagram, Threads)
- Offrir une vision analytique complete des performances de publication
- Garantir la gouvernance, la securite et la tracabilite des actions

1.2 Principes directeurs

Incrementalisme maitrise : Chaque fonctionnalite V2 est un module autonome qui peut etre active independamment. L'existant V1 ne doit jamais etre casse par un ajout. Les migrations de base de donnees sont reversibles et les anciennes donnees sont preservees integralement.

API-first : Toute fonctionnalite est exposee via une API REST clairement documentee. Le frontend consomme exclusivement ces API, ce qui facilite les tests, le versioning et la future mise en place d'applications mobiles ou d'integrations tierces.

Experience utilisateur premium : L'interface doit etre intuitive, reactive et coherente. Le dark mode est supporte nativement. Les interactions sont fluides grace a des optimisations de performance (server

components, streaming, optimistic updates).

Securite et conformite : Les tokens OAuth sont chiffres en base de donnees, les acces sont journalises, les roles sont fins et les actions sensibles necessitent une re-authentification. La plateforme est prete pour un audit de securite externe.

2. Architecture technique V2

L'architecture V2 conserve le monolithe modulaire de la V1 (Next.js App Router) mais introduit des couches de services dediees pour chaque domaine fonctionnel. Ce choix architectural privilegie la simplicite de deploiement tout en preparant une future extraction en microservices si le trafic l'exige. Le schema suivant presente la vue d'ensemble des couches techniques de la plateforme.

2.1 Stack technique

Le choix technologique reste coherent avec la V1, avec des evolutions ciblees pour repondre aux besoins de scalabilite et de robustesse requis par la V2. Les principaux changements concernent la migration de SQLite vers PostgreSQL pour la production, l'ajout d'un scheduler pour les publications differees, et l'introduction d'un systeme de files de messages pour les notifications asynchrones.

Couche	Technologie V1	Technologie V2	Raison du changement
Framework	Next.js 16 App Router	Next.js 16 App Router	Continuite, RSC matures
Base de donnees	SQLite (Prisma)	PostgreSQL (Prisma)	Scalabilite, concurrency
ORM	Prisma 6	Prisma 6+	Requetes complexes, migrations
Authentification	JWT (jose)	JWT (jose) + refresh	Securite renforcee
Etat frontend	Zustand	Zustand + TanStack Query	Cache serveur, optimistic UI
UI	shadcn/ui + Tailwind 4	shadcn/ui + Tailwind 4	Continuite
Scheduler	Aucun	BullMQ + Redis	Publications differees, cron
Stockage fichiers	Aucun	S3-compatible (MinIO)	Medias, exports
Notifications	Aucun	Email (Resend) + Slack Webhook	Alertes, rappels
Drag and Drop	dnd-kit	dnd-kit + calendar lib	Calendrier editorial
Charts	Recharts	Recharts + Visx	Analytics avances

Tableau 1 - Comparaison des stacks techniques V1 vs V2

2.2 Architecture en couches

L'architecture V2 s'organise en quatre couches principales, chacune responsable d'un domaine fonctionnel distinct. Cette separation permet une evolution independante des modules et facilite les tests unitaires et d'integration. La couche de presentation gere l'interface utilisateur et les interactions. La couche API expose les endpoints REST et gere l'authentification. La couche service encapsule la logique metier. La couche donnees gere l'accès aux donnees via Prisma et le systeme de fichiers.

Couche	Composants	Responsabilites
Presentation	React Components, Zustand, TanStack Query	Rendu UI, gestion d'etat client, cache, drag and drop, calendrier
API Routes	Next.js App Router API handlers	Authentification, validation entrees, orchestration services
Services	AI Service, Publishing Service, Analytics Service, Notification Service	Logique metier, appels IA, publication reseaux, calculs analytics
Donnees	Prisma Client, PostgreSQL, S3/MinIO, Redis	Persistance, requetes, stockage fichiers, file de messages

Tableau 2 - Architecture en couches de la V2

2.3 Diagramme d'architecture globale

Le schema ci-dessous presente le flux de donnees principal de la plateforme, depuis la creation d'un post jusqu'a sa publication sur les reseaux sociaux. Chaque bloc represente un module fonctionnel qui communique avec les autres via des appels de fonction internes (dans le cadre du monolithe) ou via des API REST. Les flux asynchrone (notifications, publications differees) passent par le scheduler BullMQ.

Utilisateur	Interface Web	API Routes	Services Metier	Base de donnees + S3
Createur	Formulaire post	POST /api/posts	PostService.create()	Post, AIVariant
Editeur	Calendrier drag/drop	PUT /api/posts/:id/schedule	SchedulerService.schedule()	Redis (BullMQ)
IA	Onglet generation	POST /api/posts/:id/generate	AIService.generate()	AIVariant, PromptTemplate
Valideur	Workflow approbation	POST /api/posts/:id/validate	ValidationService.validate()	ValidationStep, AuditLog
Admin	Dashboard analytics	GET /api/analytics	AnalyticsService.compute()	Post, PublicationLog agr.
Systeme	Publication auto	BullMQ worker	PublishingService.publish()	SocialAccount, PubLog
Systeme	Notification	BullMQ worker	NotificationService.send()	Notification, EmailLog

Tableau 3 - Flux de donnees principal de la plateforme V2

2.4 Gestion des environnements

La V2 introduit une gestion fine des environnements avec configuration par variable d'environnement. Trois environnements sont supportes : development (local), staging (pre-production) et production. Chaque environnement dispose de sa propre base de donnees PostgreSQL, de son bucket S3 et de ses tokens OAuth. Les variables sensibles (JWT_SECRET, API_KEYS, DB_URL) sont geres via un vault ou un service de secrets et ne sont jamais commitees dans le depot de code.

Variable	Description	Exemple
DATABASE_URL	URL de connexion PostgreSQL	postgresql://user:pass@host:5432/db
REDIS_URL	URL de connexion Redis	redis://host:6379
S3_ENDPOINT	Endpoint S3-compatible	https://s3.amazonaws.com
JWT_SECRET	Cle de signature JWT	256-bit random key
LINKEDIN_CLIENT_ID	Client ID OAuth LinkedIn	li_xxx
SLACK_WEBHOOK_URL	Webhook Slack pour notifications	https://hooks.slack.com/...
SMTP_URL	Configuration email (Resend)	smtp://resend:xxx
NODE_ENV	Environnement courant	development staging production

Tableau 4 - Variables d'environnement de la V2

3. Schema de base de donnees V2

Le schema de donnees V2 etend significativement celui de la V1 pour supporter les nouvelles fonctionnalites de calendrier editorial, de branding, de bibliotheque de prompts, de gestion multi-marques, de media library, d'analytics et de gouvernance avancee. La migration de SQLite vers PostgreSQL permet d'exploiter les types avances (JSONB, arrays), les contraintes d'integrite referentielle et les performances superieures en environnement multi-utilisateurs.

Le schema V2 comporte 24 modeles au total, contre 7 en V1. Chaque nouveau modele est concu pour repondre a un besoin fonctionnel specifique tout en s'integrant dans un ecosysteme relationnel coherent. Les relations sont materialisees par des foreign keys avec cascades appropriees pour garantir l'integrite referentielle.

3.1 Nouveaux modeles

3.1.1 Workspace (Espace de travail)

Le modele Workspace est le coeur de la gestion multi-marques. Chaque workspace represente un espace de travail isole contenant ses propres posts, parametres de branding, pages sociales et membres. Cela permet a une agence ou une grande entreprise de gerer plusieurs marques ou departments depuis la meme instance de l'application, avec une separation complete des donnees et des configurations.

Champ	Type	Contraintes	Description
id	String (cuid)	PK	Identifiant unique
name	String	NOT NULL	Nom de l'espace (ex: "DataSphere Innovation")
slug	String	UNIQUE	Slug URL-friendly
description	String?	NULLABLE	Description de l'espace
logoUrl	String?	NULLABLE	URL du logo
isActive	Boolean	default: true	Espace actif
createdAt / updatedAt	DateTime	auto	Timestamps

Tableau 5 - Schema du modele Workspace

3.1.2 BrandProfile (Branding editorial)

Le BrandProfile encapsule l'identite editoriale d'une marque au sein d'un workspace. Il definit le ton de la communication, les mots a utiliser et a eviter, les CTA preferes, les hashtags de marque et les templates de structure des posts. Ces informations sont injectees automatiquement dans les prompts IA pour garantir une coherence de ton dans l'ensemble des publications generees.

Champ	Type	Description
id / workspaceId	String / FK	Identifiant + lien vers Workspace
brandTone	String	Ton de marque (ex: "expert", "decontracte")
brandVoice	String	Voix de la marque (ex: "professionnel mais accessible")
targetAudience	String	Audience cible par default
preferredLanguage	String	Langue principale (ex: "fr")
wordsToUse	Json	Liste de mots a valoriser
wordsToAvoid	Json	Liste de mots a proscrire
favoriteCtas	Json	CTA favoris avec intitules
brandHashtags	Json	Hashtags de marque a inclure

postTemplates	Json	Templates de structure de posts
---------------	------	---------------------------------

Tableau 6 - Schema du modele BrandProfile

3.1.3 PromptTemplate (Bibliotheque de prompts)

La bibliotheque de prompts centralise tous les system prompts utilises pour la generation de contenu IA. Chaque template est categorise par type de post (expertise, storytelling, carousel, teaser, video, institutionnel) et peut etre personnalise par les administrateurs. Les editeurs peuvent selectionner le template adapte lors de la creation d'un post, et le systeme injecte automatiquement le branding du workspace courant dans le prompt.

Champ	Type	Description
id	String (cuid)	Identifiant unique
workspaceId	FK -> Workspace	Espace de travail propriétaire
name	String	Nom du template (ex: "Expertise IA")
type	Enum	expertise storytelling carousel teaser video institutionnel
systemPrompt	String	Prompt system complet
variables	Json	Variables dynamiques injectables
isDefault	Boolean	Template par défaut pour ce type
isActive	Boolean	Template actif
version	Int	Numero de version pour historique
createdAt / updatedAt	DateTime	Timestamps

Tableau 7 - Schema du modele PromptTemplate

3.1.4 Media (Bibliotheque mediatique)

Le modele Media gere la bibliotheque de fichiers mediatiques (images, videos, documents) associes aux posts et aux workspaces. Les fichiers sont stockes dans un bucket S3-compatible (MinIO en dev, AWS S3 en prod) et les metadonnees sont persistees en base de donnees. Le systeme supporte l'upload, la compression, le redimensionnement automatique et la generation de thumbnails.

Champ	Type	Description
id	String (cuid)	Identifiant unique
workspaceId	FK -> Workspace	Espace de travail
uploadedBy	FK -> User	Utilisateur uploadur
originalName	String	Nom original du fichier
mimeType	String	Type MIME (image/png, video/mp4...)
size	Int	Taille en octets
url	String	URL d'accès S3
thumbnailUrl	String?	URL de la miniature
width / height	Int?	Dimensions en pixels
alt	String?	Texte alternatif
tags	Json	Tags de recherche

Tableau 8 - Schema du modele Media

3.1.5 PostMedia (Association post-medias)

Le modele PostMedia est une table de liaison many-to-many entre Post et Media, permettant d'associer plusieurs fichiers mediatiques a un meme post. Le champ position definit l'ordre d'affichage (pour les carrousels par exemple) et le champ role indique le type d'association (image principale, carrousel, thumbnail video, document joint).

Champ	Type	Description
id	String (cuid)	Identifiant unique
postId	FK -> Post	Post associe
mediaId	FK -> Media	Media associe
position	Int	Ordre d'affichage
role	Enum	hero carrousel thumbnail attachment
autoGenerated	Boolean	Genere automatiquement par IA

Tableau 9 - Schema du modele PostMedia

3.1.6 ContentScore (Scores de qualite)

Le modele ContentScore stocke les metriques de qualite calculees pour chaque variante de post generee par l'IA. Trois scores sont evalues : qualite redactionnelle, lisibilite et potentiel d'engagement. Ces scores sont calcules par des modeles d'evaluation specifiques ou par des heuristiques basees sur la structure du contenu (longueur, presence de CTA, usage de hashtags, etc.).

Champ	Type	Description
id	String (cuid)	Identifiant unique
aiVariantId	FK -> AIVariant	Variante evaluee
qualityScore	Float	Score qualite (0-100)
readabilityScore	Float	Score lisibilite (0-100)
engagementScore	Float	Score potentiel engagement (0-100)
suggestions	Json	Suggestions d'amelioration
metrics	Json	Metriques detaillees par critere

Tableau 10 - Schema du modele ContentScore

3.1.7 CalendarEvent (Evenements calendrier)

Le modele CalendarEvent represente les entrees du calendrier editorial. Chaque evenement est lie a un post et porte les informations de planification (date, duree, recurrence). Les evenements peuvent etre deplaces par drag and drop dans l'interface, ce qui met a jour automatiquement la date de publication prevue du post associe. Le champ eventType permet de distinguer les posts planifies des evenements recurrents (jalons editoriaux, campagnes, etc.).

Champ	Type	Description
id	String (cuid)	Identifiant unique
workspaceId	FK -> Workspace	Espace de travail
postId	FK -> Post?	Post associe (optionnel pour jalons)
title	String	Titre de l'evenement
startDate	DateTime	Date et heure de debut
endDate	DateTime?	Date et heure de fin

eventType	Enum	post milestone campaign recurring
recurrenceRule	String?	Regle de recurrence (RFC 5545)
color	String?	Couleur d'affichage

Tableau 11 - Schema du modele CalendarEvent

3.1.8 SocialAccount (Comptes sociaux multi-reseaux)

Le modele SocialAccount remplace et generalise le modele LinkedInAccount de la V1. Il supporte la connexion de comptes sur plusieurs reseaux sociaux (LinkedIn, X/Twitter, Facebook, Instagram, Threads) et stocke les tokens OAuth chiffres pour chaque plateforme. La migration des LinkedInAccount existants vers ce nouveau modele est automatique et transparente.

Champ	Type	Description
id	String (cuid)	Identifiant unique
workspaceId	FK -> Workspace	Espace de travail
platform	Enum	linkedin twitter facebook instagram threads
platformUserId	String	ID utilisateur sur la plateforme
displayName	String	Nom d'affichage du compte
accessToken	String	Token OAuth (chiffre)
refreshToken	String?	Token de rafraichissement
tokenExpiresAt	DateTime?	Date d'expiration du token
scope	String?	Permissions accordees
isActive	Boolean	Compte actif
metadata	Json?	Metadonnees specifiques au reseau

Tableau 12 - Schema du modele SocialAccount

3.1.9 ValidationStep (Workflow multi-approbateurs)

Le modele ValidationStep represente les etapes successives du workflow de validation avance. Contrairement a la V1 ou l'approbation etait mono-etape, la V2 permet de definir des workflows multi-etapes avec des approbateurs differents a chaque etape. Par exemple : revue editoriale, validation juridique, approbation direction. Chaque etape doit etre validee avant de passer a la suivante.

Champ	Type	Description
id	String (cuid)	Identifiant unique
postId	FK -> Post	Post concerne
stepOrder	Int	Ordre de l'etape
stepName	String	Nom de l'etape (ex: "Revue editoriale")
requiredRole	String	Role requis pour cette etape
assignedUserId	FK -> User?	Approbateur designe
status	Enum	pending approved rejected skipped
comment	String?	Commentaire de l'approbateur
resolvedAt	DateTime?	Date de resolution

Tableau 13 - Schema du modele ValidationStep

3.1.10 Notification

Le modele Notification centralise la gestion des notifications envoyees aux utilisateurs. Les notifications peuvent etre de type email, Slack ou in-app. Elles sont declenchees par des evenements specifiques : post en attente de validation, approbation/rejet, echec de publication, rappel de planning, etc. Le systeme utilise BullMQ pour les envois asynchrones et supporte les retries avec backoff exponentiel.

Champ	Type	Description
id	String (cuid)	Identifiant unique
userId	FK -> User	Destinataire
type	Enum	email slack in_app
channel	String	Canal d'envoi (ex: "validation_pending")
title	String	Titre de la notification

message	String	Contenu du message
isRead	Boolean	Notification lue (in-app)
sentAt	DateTime?	Date d'envoi effectif
readAt	DateTime?	Date de lecture

Tableau 14 - Schema du modele Notification

3.2 Modeles modifies (evolution V1)

Plusieurs modeles existants de la V1 sont modifies pour supporter les nouvelles fonctionnalites. Le modele Post recoit les nouveaux champs workspaceId, contentFormat, carouselData et scriptVideo pour supporter les formats enrichis. Le modele User recoit les champs workspaceIds (relation many-to-many avec Workspace) et permissions (JSON) pour la gestion fine des roles. Le modele AIVariant est etendu avec le champ variantType pour distinguer les variantes standards des variantes carrousel et video.

Modele	Champs ajoutes	Description
Post	workspaceId, contentFormat, carouselData, scriptVideo, contentScores	Lien workspace, format enrichi, donnees carrousel, script video
User	workspaceIds, permissions, avatarUrl, lastLoginAt	Multi-workspace, permissions fines, avatar, tracking
AIVariant	variantType, carouselSlides, scores	Type variante (standard/carrousel/video), slides carrousel
AuditLog	workspaceId, ipAddress, userAgent, severity	Contexte workspace, metadata securite, niveau de gravite
Settings	workspaceId, category	Settings par workspace, categorisation

Tableau 15 - Modifications des modeles existants

3.3 Relations et diagramme ER

Le diagramme suivant presente les principales relations entre les 24 modeles du schema V2. Les relations one-to-many et many-to-many sont materialisees par des foreign keys. Les cascades de suppression sont configurees pour garantir l'integrite referentielle : par exemple, la suppression d'un workspace entraine la suppression de tous ses posts, medias et parametres en cascade.

Entite source	Relation	Entite cible	Cardinalite
Workspace	contient	BrandProfile	1:1
Workspace	contient	Post	1:N
Workspace	contient	Media	1:N

Workspace	contient	CalendarEvent	1:N
Workspace	contient	SocialAccount	1:N
Workspace	contient	PromptTemplate	1:N
Workspace	contient	User (M:N)	M:N (WorkspaceMember)
Post	contient	AIVariant	1:N
Post	contient	ValidationStep	1:N
Post	contient	PostMedia	1:N
Post	contient	PublicationLog	1:N
Post	associe a	SocialAccount	N:1
User	cree	Post	1:N
User	valide	ValidationStep	1:N
User	recut	Notification	1:N
AIVariant	possede	ContentScore	1:1

Tableau 16 - Relations principales du schema V2

4. Nouveaux ecrans

L'interface V2 s'enrichit de six nouveaux ecrans majeurs, en plus des ameliorations apportees aux ecrans existants. Le routing evolue du systeme Zustand interne vers le routing natif Next.js App Router pour profiter du SSR, du prefetching et de la navigation par URL. Chaque ecran est concu comme un composant modulaire avec ses propres sous-composants et son propre chargement de donnees via TanStack Query.

4.1 Vue calendrier editorial

Le calendrier editorial est l'ecran central de la V2. Il offre trois vues : mensuelle, hebdomadaire et liste. Les posts planifies apparaissent sous forme de cartes de couleur differente selon leur statut (brouillon en jaune, en attente en orange, approuve en vert, publie en bleu). Le drag and drop permet de deplacer un post d'une date a une autre, ce qui met a jour automatiquement le scheduledDate. Un panneau lateral affiche le detail du post selectionne avec la possibilite de le modifier directement.

- **Vues** : Mois / Semaine / Liste avec switch interactif
- **Drag and Drop** : Deplacement de posts entre dates via dnd-kit
- **Filtres** : Par statut, auteur, type de contenu, reseau social
- **Quick create** : Creation rapide de post depuis un clic sur une date vide
- **File d'attente** : Panneau lateral avec posts non planifies a glisser dans le calendrier

- **Jalons** : Affichage de jalons editoriaux (campagnes, evenements)

4.2 Bibliotheque de prompts

L'ecran de gestion des prompts permet aux administrateurs de creer, modifier et organiser les templates de prompts utilises pour la generation de contenu IA. L'interface presente une grille de cartes, chaque carte representant un template avec son nom, son type, son statut (actif/inactif) et un aperçu du prompt. Un editeur de prompt avec coloration syntaxique et preview de la generation est integre directement.

- **Liste/Grid** : Basculement entre vue liste et grille de cartes
- **Editeur** : Editeur de texte riche pour les prompts avec variables dynamiques
- **Aperçu** : Preview de generation IA directement dans l'editeur
- **Categories** : Filtrage par type (expertise, storytelling, carousel, teaser, video, institutionnel)
- **Import/Export** : Import et export de templates au format JSON

4.3 Branding editorial

L'ecran de branding editoriale permet de configurer l'identite de marque de chaque workspace. Il se presente sous forme de sections organisees : ton et voix, vocabulaire (mots a utiliser/eviter), CTA favoris, hashtags de marque, et templates de structure. Un assistant IA propose des suggestions de branding basees sur les posts deja publies et le secteur d'activite de l'entreprise.

- **Ton et voix** : Selection de ton (expert, decontracte, inspirant, provocateur)
- **Vocabulaire** : Listes de mots a utiliser et a eviter avec ajout/suppression dynamique
- **CTA** : Librairie de CTA favoris avec compteur d'utilisation
- **Hashtags** : Listes de hashtags de marque avec suggestions automatiques
- **Templates** : Modeles de structure (hook + developpement + conclusion + CTA)

4.4 Media Library

La bibliotheque mediatique offre une interface de type "explorateur de fichiers" pour gerer les images, videos et documents associes aux posts. Elle supporte l'upload multiple avec drag and drop, la recherche par tags, le filtrage par type et par date, et la visualisation en grille ou en liste. Un editeur d'image integre permet le recadrage et le redimensionnement directement dans l'interface.

- **Upload** : Drag and drop multi-fichiers avec barre de progression
- **Grille/Liste** : Double vue avec tri par date, nom, taille
- **Recherche** : Recherche plein texte + filtrage par tags et type MIME
- **Association** : Association automatique image/post basee sur les tags
- **Suggestions** : Generation de visuels par IA (text-to-image)

4.5 Dashboard analytics

Le dashboard analytics remplace le dashboard V1 basique par une suite complete de graphiques et metriques. Il presente un resume global en haut de page (KPI cards), suivi de graphiques detailles : frequence de publication par semaine/mois, taux de publication reussie par reseau, classement des meilleurs angles de contenu, comparaison des performances par provider IA, et historique des publications. Les donnees sont mise a jour en temps reel via TanStack Query avec rafraichissement automatique.

- **KPI Cards** : Posts publies ce mois, taux de reussite, engagement moyen, temps moyen de validation
- **Frequence** : Graphique en barres de la frequence de publication par semaine
- **Taux de reussite** : Donut chart par reseau social
- **Meilleurs angles** : Classement des angles par score d'engagement moyen
- **Comparaison IA** : Radar chart comparant les performances par provider
- **Historique** : Timeline des publications avec filtres multi-criteres

4.6 Gestion des workspaces

L'ecran de gestion des workspaces permet aux administrateurs de creer, configurer et gerer les espaces de travail. Chaque workspace est presente sous forme de carte avec son logo, son nom et le nombre de membres. Un drawer lateral permet d'acceder aux parametres detailles du workspace : informations generales, membres et roles, comptes sociaux connectes, parametres de branding, et statistiques d'utilisation.

- **Liste workspaces** : Grille de cartes avec logo et stats rapides
- **Switcher** : Selecteur de workspace courant dans la barre de navigation
- **Parametres** : Drawer lateral avec sections organisees par onglets
- **Membres** : Gestion des membres avec invitation par email
- **Comptes sociaux** : Connexion/deconnexion multi-reseaux par workspace

5. Nouvelles API routes

La V2 introduit 32 nouvelles API routes, portant le total a environ 46 endpoints. Chaque endpoint suit les conventions REST : noms de ressources au pluriel, methodes HTTP semantiques (GET pour lire, POST pour creer, PUT pour modifier, DELETE pour supprimer), codes de statut HTTP explicites et reponses JSON structurees. L'authentification JWT est obligatoire sur tous les endpoints sauf login/register et health check.

5.1 APIs Calendrier

GET	/api/calendar/events	Lister les evenements du calendrier avec filtres (dateDebut, dateFin, type)
POST	/api/calendar/events	Creer un evenement calendrier (post planifie ou jalon)

PUT	/api/calendar/events/:id	Modifier un evenement (deplacement, changement de date)
PUT	/api/calendar/events/:id/move	Deplacer un evenement par drag and drop
DELETE	/api/calendar/events/:id	Supprimer un evenement
GET	/api/calendar/queue	Lister les posts non planifies (file d'attente)
POST	/api/calendar/schedule	Planifier un post (creer evenement + mettre a jour scheduledDate)

Tableau 17 - APIs Calendrier

5.2 APIs Prompts

GET	/api/prompts	Lister les templates de prompts du workspace courant
POST	/api/prompts	Creer un nouveau template de prompt
GET	/api/prompts/:id	Recuperer un template par ID
PUT	/api/prompts/:id	Modifier un template existant
DELETE	/api/prompts/:id	Desactiver un template (soft delete)
POST	/api/prompts/:id/preview	Generer un apercu avec le template
POST	/api/prompts/import	Importer des templates depuis JSON
GET	/api/prompts/export	Exporter tous les templates du workspace

Tableau 18 - APIs Prompts

5.3 APIs Branding

GET	/api/branding	Recuperer le profil de marque du workspace courant
PUT	/api/branding	Mettre a jour le profil de marque complet
PATCH	/api/branding/tone	Modifier le ton et la voix
PATCH	/api/branding/vocabulary	Modifier les listes de mots
PATCH	/api/branding/ctas	Modifier les CTA favoris
PATCH	/api/branding/hashtags	Modifier les hashtags de marque
POST	/api/branding/suggest	Obtenir des suggestions IA de branding

5.4 APIs Medias

GET	/api/media	Lister les medias du workspace (pagination, filtres)
POST	/api/media/upload	Uploader un ou plusieurs fichiers (multipart/form-data)
GET	/api/media/:id	Recuperer un media par ID
PUT	/api/media/:id	Modifier les metadonnees d'un media
DELETE	/api/media/:id	Supprimer un media (fichier S3 + BDD)
POST	/api/media/:id/suggest-posts	Suggerer des posts associes a ce media
POST	/api/media/generate-visual	Generer un visuel par IA (text-to-image)

Tableau 20 - APIs Medias

5.5 APIs Analytics

GET	/api/analytics/overview	Resume global (KPI cards)
GET	/api/analytics/frequency	Frequence de publication par periode
GET	/api/analytics/success-rate	Taux de reussite par reseau social
GET	/api/analytics/top-angles	Classement des meilleurs angles de contenu
GET	/api/analytics/provider-comparison	Comparaison des performances par provider IA
GET	/api/analytics/history	Historique des publications avec pagination
GET	/api/analytics/export	Export des donnees analytics (CSV/JSON)

Tableau 21 - APIs Analytics

5.6 APIs Workspaces

GET	/api/workspaces	Lister les workspaces accessibles a l'utilisateur
POST	/api/workspaces	Creer un nouveau workspace
GET	/api/workspaces/:id	Recuperer un workspace par ID
PUT	/api/workspaces/:id	Modifier les informations du workspace

DELETE	/api/workspaces/:id	Desactiver un workspace
GET	/api/workspaces/:id/members	Lister les membres du workspace
POST	/api/workspaces/:id/members	Ajouter un membre au workspace
PUT	/api/workspaces/:id/members/:userId	Modifier le role d'un membre
DELETE	/api/workspaces/:id/members/:userId	Retirer un membre
POST	/api/workspaces/switch	Changer de workspace courant

Tableau 22 - APIs Workspaces

5.7 APIs Reseaux sociaux (multi-reseaux)

GET	/api/social-accounts	Lister les comptes sociaux du workspace
POST	/api/social-accounts/connect	Connecter un nouveau compte social (OAuth)
DELETE	/api/social-accounts/:id	Deconnecter un compte social
GET	/api/social-accounts/:id/status	Verifier la validite du token OAuth
POST	/api/social-accounts/:id/refresh	Rafraichir le token OAuth expir
POST	/api/publish/multi	Publier un post sur plusieurs reseaux simultanement

Tableau 23 - APIs Reseaux sociaux

5.8 APIs Validation avancee

GET	/api/posts/:id/steps	Lister les etapes de validation d'un post
POST	/api/posts/:id/steps	Creer un workflow de validation multi-etapes
PUT	/api/posts/:id/steps/:stepId	Approuver, rejeter ou sauter une etape
GET	/api/validation/queues	Lister les posts en attente de validation pour l'utilisateur
POST	/api/validation/bulk	Approbation/rejet en masse de plusieurs posts

Tableau 24 - APIs Validation avancee

5.9 APIs Notifications

GET	/api/notifications	Lister les notifications de l'utilisateur
PUT	/api/notifications/:id/read	Marquer une notification comme lue
PUT	/api/notifications/read-all	Marquer toutes les notifications comme lues
GET	/api/notifications/unread-count	Compteur de notifications non lues
PUT	/api/settings/notifications	Configurer les preferences de notification

Tableau 25 - APIs Notifications

6. Code et pseudo-code detaille

Cette section presente les implementations cles de la V2 sous forme de code TypeScript simplifie. Le focus est porte sur les services principaux : service de generation IA enrichie, service de publication multi-reseaux, service de calendrier avec drag and drop, et service de scoring de qualite du contenu. Chaque extrait est accompagne d'explications sur les choix d'implementation et les patterns utilises.

6.1 Schema Prisma V2 - Extrait

L'extrait suivant presente les nouveaux modeles du schema Prisma V2. Les types PostgreSQL comme JsonB sont utilises pour les champs semi-structures (wordsToUse, carouselSlides, etc.). Les enums sont definis au niveau du schema pour garantir la coherence des donnees. Les contraintes d'unicite et les index sont ajoutes pour optimiser les requetes frequentes.

```
// prisma/schema.prisma (extrait V2)

model Workspace {
  id          String    @id @default(cuid())
  name        String
  slug        String    @unique
  logoUrl     String?
  isActive    Boolean   @default(true)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  posts       Post[]
  medias       Media[]
  socialAccounts SocialAccount[]
  brandProfile BrandProfile?
  promptTemplates PromptTemplate[]
  members      WorkspaceMember[]
}

model BrandProfile {
  id          String    @id @default(cuid())
  workspaceId String    @unique
  workspace   Workspace @relation(fields: [workspaceId])
  brandTone   String    @default("professional")
  brandVoice  String    @default("")
}
```

```

wordsToUse      Json    @default("[]")
wordsToAvoid    Json    @default("[]")
favoriteCtas    Json    @default("[]")
brandHashtags   Json    @default("[]")
postTemplates   Json    @default("[]")
}

```

6.2 Service de generation IA enrichie

Le service d'IA enrichi etend celui de la V1 en ajoutant la generation de carrousels, de scripts video, de variantes courtes/longues et le scoring automatique de qualite. Le service injecte automatiquement le branding du workspace courant dans les prompts et utilise les templates de la bibliotheque pour structurer la generation.

```

// src/lib/services/ai-service.ts

interface EnrichedGenerationOptions {
  postId: string;
  workspaceId: string;
  promptTemplateId?: string;
  contentFormat: 'standard' | 'carousel' | 'video' | 'short' | 'long';
  provider: AIProvider;
  targetAudience?: string;
  angle?: string;
}

async function generateEnrichedContent(opts: EnrichedGenerationOptions) {
  // 1. Charger le branding du workspace
  const brand = await prisma.brandProfile.findUnique({
    where: { workspaceId: opts.workspaceId }
  });

  // 2. Charger le template de prompt
  let systemPrompt = DEFAULT_SYSTEM_PROMPT;
  if (opts.promptTemplateId) {
    const tpl = await prisma.promptTemplate.findUnique({
      where: { id: opts.promptTemplateId }
    });
    if (tpl) systemPrompt = tpl.systemPrompt;
  }

  // 3. Injecter le branding dans le prompt
  const enrichedPrompt = buildPromptWithBranding({
    systemPrompt,
    brandTone: brand?.brandTone ?? "professional",
    wordsToUse: brand?.wordsToUse as string[] ?? [],
    wordsToAvoid: brand?.wordsToAvoid as string[] ?? [],
    hashtags: brand?.brandHashtags as string[] ?? [],
    format: opts.contentFormat,
    audience: opts.targetAudience,
    angle: opts.angle,
  });
}

```

```

// 4. Generer les variantes selon le format
const variants = await generateVariants(enrichedPrompt, opts.provider, {
  format: opts.contentFormat,
  count: opts.contentFormat === 'carousel' ? 1 : 3,
});

// 5. Calculer les scores de qualite
for (const v of variants) {
  const scores = await calculateContentScores(v.content);
  await prisma.contentScore.create({
    data: { aiVariantId: v.id, ...scores }
  });
}

return variants;
}

```

6.3 Service de scoring de qualite

Le service de scoring evalue la qualite de chaque variante generee selon trois dimensions : qualite redactionnelle (structure, coherence, richesse du vocabulaire), lisibilite (longueur des phrases, complexite lexicale, presence de CTA) et potentiel d'engagement (accroche, emotionnalite, appel a l'action). Les scores sont combines en un score global et des suggestions d'amelioration sont generees.

```

// src/lib/services/scoring-service.ts

interface ContentScores {
  qualityScore: number; // 0-100
  readabilityScore: number; // 0-100
  engagementScore: number; // 0-100
  suggestions: string[];
  metrics: Record<string, number>;
}

async function calculateContentScores(content: string): Promise<ContentScores> {
  const metrics: Record<string, number> = {};
  const suggestions: string[] = [];

  // Qualite redactionnelle
  metrics.charCount = content.length;
  metrics.sentenceCount = content.split(/[!?!]+/).filter(Boolean).length;
  metrics.avgSentenceLength = metrics.charCount / metrics.sentenceCount;
  metrics.paragraphCount = content.split(/\n\n+/).filter(Boolean).length;
  metrics.hasCta = content.match(/(contactez|decouvrez|partagez|commentez)/i) ? 1 : 0;
  metrics.hasHashtags = content.match(/#\w+/) ? 1 : 0;

  // Score de qualite (ponderation)
  let qualityScore = 50;
  if (metrics.charCount >= 500) qualityScore += 10;
  if (metrics.avgSentenceLength >= 10 && metrics.avgSentenceLength <= 25) qualityScore += 15;
}

```



```

    if (metrics.hasCta) qualityScore += 15;
    if (metrics.paragraphCount >= 3) qualityScore += 10;
    qualityScore = Math.min(100, qualityScore);

    if (!metrics.hasCta) suggestions.push("Ajoutez un appel a l'action (CTA)");
    if (metrics.charCount < 300) suggestions.push("Enrichissez le contenu (min. 500 car.)");

    // Lisibilite (indice de lisibilite simplifie)
    let readabilityScore = 70;
    if (metrics.avgSentenceLength < 15) readabilityScore += 20;
    else if (metrics.avgSentenceLength > 30) readabilityScore -= 20;
    if (metrics.paragraphCount >= 4) readabilityScore += 10;
    readabilityScore = Math.max(0, Math.min(100, readabilityScore));

    // Potentiel d'engagement (heuristiques)
    let engagementScore = 40;
    const hasHook = /^(?|Imagine|Saviez-vous|En |Pourquoi)/.test(content.trim());
    if (hasHook) engagementScore += 25;
    const hasQuestion = content.includes("?");
    if (hasQuestion) engagementScore += 15;
    if (metrics.hasHashtags) engagementScore += 10;
    const hasNumbers = /\d+/.test(content);
    if (hasNumbers) engagementScore += 10;
    engagementScore = Math.min(100, engagementScore);

    if (!hasHook) suggestions.push("Commencez par une question ou un accroche");
    if (!hasQuestion) suggestions.push("Posez une question pour susciter l'engagement");

    return { qualityScore, readabilityScore, engagementScore, suggestions, metrics };
}

```

6.4 Service de publication multi-reseaux

Le service de publication multi-reseaux est conçu pour être extensible. Chaque réseau social est implémenté sous forme d'adaptateur respectant une interface commune. Cela permet d'ajouter de nouveaux réseaux sans modifier le code existant (principe Open/Closed). Le service gère le retry automatique en cas d'échec, le logging des publications et la mise à jour du statut du post.

```

// src/lib/services/publishing-service.ts

interface SocialPublisher {
  publish(content: string, media: MediaItem[]): Promise<PublishResult>;
  refreshToken(account: SocialAccount): Promise<void>;
  verifyToken(account: SocialAccount): Promise<boolean>;
}

class LinkedInPublisher implements SocialPublisher {
  async publish(content, media) {
    // Appel API LinkedIn v2
    const response = await fetch('https://api.linkedin.com/v2/ugcPosts', {

```

```

        method: 'POST',
        headers: { Authorization: `Bearer ${this.token}` },
        body: JSON.stringify({ author: this.orgId, text: content })
    });
    return { platform: 'linkedin', postId: response.id, status: 'success' };
}
}

class PublishingService {
    private publishers = new Map<string, SocialPublisher>();

    constructor() {
        this.publishers.set('linkedin', new LinkedInPublisher());
        // Futur: twitter, facebook, instagram, threads
    }

    async publishToMultiple(
        postId: string,
        platforms: string[]
    ): Promise<Map<string, PublishResult>> {
        const post = await prisma.post.findUnique({
            where: { id: postId },
            include: { postMedias: { include: { media: true } } }
        });

        const results = new Map();
        for (const platform of platforms) {
            try {
                const publisher = this.publishers.get(platform);
                const media = post.postMedias.map(pm => pm.media);
                const result = await publisher.publish(post.finalContent, media);
                results.set(platform, result);
            } catch (error) {
                results.set(platform, {
                    platform, status: 'failed', error: error.message
                });
            }
        }

        // Logger les resultats
        await logPublishResults(postId, results);
        return results;
    }
}

```

6.5 Service de calendrier (drag and drop)

Le service de calendrier gere les operations de deplacement d'evenements via drag and drop. Lorsqu'un utilisateur deplace un post dans le calendrier, le service met a jour simultanement l'evenement calendrier et la date de publication prevue du post. Les operations sont transactionnelles pour garantir la coherence des donnees. Si l'evenement est un jalon (sans post associe), seule la date de l'evenement est mise a jour.

```
// src/app/api/calendar/events/[id]/move/route.ts

export async function PUT(request: Request, { params }: { params: { id: string } }) {
  const user = await getAuthUser(request);
  if (!user) return NextResponse.json({ error: 'Non autorise' }, { status: 401 });

  const { newStartDate, newEndDate } = await request.json();
  const eventId = params.id;

  // Transaction : mettre a jour evenement + post
  const result = await prisma.$transaction(async (tx) => {
    const event = await tx.calendarEvent.update({
      where: { id: eventId },
      data: { startDate: new Date(newStartDate),
        endDate: newEndDate ? new Date(newEndDate) : null }
    });

    // Si l'evenement est lie a un post, mettre a jour scheduledDate
    if (event.postId) {
      await tx.post.update({
        where: { id: event.postId },
        data: {
          scheduledDate: new Date(newStartDate),
          status: 'scheduled'
        }
      });
    }

    await createAuditLog({
      entityType: 'CalendarEvent',
      entityId: eventId,
      action: 'move',
      userId: user.id,
      metadata: { newStartDate, newEndDate }
    });

    return event;
  });

  return NextResponse.json({ event: result });
}
```

6.6 Middleware d'authentification

La V2 introduit un middleware Next.js pour centraliser l'authentification et le controle d'accès. Ce middleware intercepte toutes les requêtes vers les routes protégées, vérifie la validité du token JWT, et injecte les informations de l'utilisateur dans les headers de la requête. Cela élimine la duplication du code d'authentification dans chaque route handler et garantit une politique de sécurité uniforme.

```
// src/middleware.ts

import { NextRequest, NextResponse } from 'next/server';
```

```

import { verifyToken } from '@/lib/auth';

const PUBLIC_PATHS = ['/api/auth/login', '/api/auth/register', '/api/health'];

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Autoriser les routes publiques
  if (PUBLIC_PATHS.some(p => pathname.startsWith(p))) {
    return NextResponse.next();
  }

  // Verifier le token JWT
  const token = request.headers.get('Authorization')?.replace('Bearer ', '');
  if (!token) {
    return NextResponse.json({ error: 'Token manquant' }, { status: 401 });
  }

  const payload = await verifyToken(token);
  if (!payload) {
    return NextResponse.json({ error: 'Token invalide' }, { status: 401 });
  }

  // Injecter les infos user dans les headers
  const requestHeaders = new Headers(request.headers);
  requestHeaders.set('x-user-id', payload.userId);
  requestHeaders.set('x-user-role', payload.role);
  requestHeaders.set('x-user-email', payload.email);

  return NextResponse.next({
    request: { headers: requestHeaders }
  });
}

export const config = {
  matcher: '/api/:path*'
};

```

6.7 Scheduler de publications differees

Le scheduler utilise BullMQ pour gerer les publications differees. Lorsqu'un post est planifie a une date future, une tache BullMQ est creee avec un delai correspondant. Un worker tourne en permanence pour traiter la file d'attente et executer les publications a l'heure prevue. En cas d'echec, le worker effectue des retries avec un backoff exponentiel (3 tentatives maximum) et notifie l'equipe en cas d'echec definitif.

```

// src/lib/services/scheduler-service.ts

import { Queue, Worker } from 'bullmq';

const publishQueue = new Queue('publish', { connection: redis });

```

```

async function schedulePost(postId: string, publishAt: Date) {
  const delay = publishAt.getTime() - Date.now();
  if (delay < 0) throw new Error('La date de publication doit etre dans le futur');

  await publishQueue.add('publish-post', { postId }, {
    delay,
    attempts: 3,
    backoff: { type: 'exponential', delay: 5000 },
    removeOnComplete: true,
    removeOnFail: false,
  });
}

// Worker (tourne en arriere-plan)
const worker = new Worker('publish', async (job) => {
  const { postId } = job.data;
  const post = await prisma.post.findUnique({
    where: { id: postId },
    include: { socialAccount: true }
  });

  if (!post || post.status !== 'scheduled') {
    return; // Post annule ou deja publie
  }

  await publishToSocialNetwork(post);
}, {
  connection: redis,
  concurrency: 5,
});

worker.on('failed', async (job, err) => {
  await prisma.post.update({
    where: { id: job.data.postId },
    data: { status: 'failed', errorMessage: err.message }
  });
  await sendNotification({
    type: 'slack',
    channel: 'publish_failed',
    message: `Publication echouee pour le post ${job.data.postId}: ${err.message}`
  });
});

```

7. Plan de migration V1 vers V2

La migration de la V1 vers la V2 est concue pour etre progressive, reversible et transparente pour les utilisateurs. Le plan se decompose en cinq phases, chacune introduisant un ensemble coherent de changements. Les phases sont ordonnees par priorite fonctionnelle et par dependance technique. Chaque phase inclut des tests de non-regression, une documentation des changements et un plan de rollback en cas de probleme.

7.1 Phase 1 - Fondations (semaines 1-3)

La première phase pose les bases techniques de la V2 sans changer l'expérience utilisateur existante. Elle comprend la migration de SQLite vers PostgreSQL, l'ajout du modèle Workspace avec la création automatique d'un workspace par défaut, l'introduction du middleware d'authentification, et la mise en place de l'infrastructure de configuration multi-environnements.

- Migration SQLite vers PostgreSQL (Prisma migrate)
- Ajout du modèle Workspace et création automatique du workspace par défaut
- Migration des LinkedInAccount vers SocialAccount (reconciliation automatique)
- Ajout du middleware Next.js pour l'authentification centralisée
- Mise en place des variables d'environnement par environnement
- Ajout de Redis pour BullMQ et le cache de session
- Tests de non-régression sur toutes les fonctionnalités V1

7.2 Phase 2 - Branding et Prompts (semaines 4-6)

La deuxième phase introduit le système de branding éditorial et la bibliothèque de prompts. Ces fonctionnalités améliorent la qualité de la génération IA sans modifier le workflow existant. Les utilisateurs peuvent désormais configurer le ton de leur marque et sélectionner des templates de prompts spécifiques lors de la génération de contenu.

- Ajout du modèle BrandProfile et de l'API /api/branding
- Ajout du modèle PromptTemplate et de l'API /api/prompts
- Intégration du branding dans le système de génération IA
- Écrans de configuration du branding et des prompts
- Migration des prompts hardcodes vers la bibliothèque
- Seed des templates de prompts par défaut

7.3 Phase 3 - Calendrier et Medias (semaines 7-10)

La troisième phase est la plus importante en termes d'impact utilisateur. Elle introduit le calendrier éditorial, la bibliothèque médiatique et la génération enrichie (carrousels, scripts vidéo, scores). Le drag and drop est implémenté pour le calendrier et l'association des médias aux posts. Le scheduler BullMQ est activé pour les publications différées.

- Ajout du modèle CalendarEvent et de l'API /api/calendar
- Écran calendrier avec vues mois/semaine/liste
- Drag and drop via dnd-kit
- Ajout du modèle Media et de l'API /api/media

- Upload S3 via MinIO (dev) et AWS S3 (prod)
- Ecran Media Library avec upload multiple
- Ajout du modele ContentScore et du service de scoring
- Generation enrichie : carrousels, scripts video, variantes courtes/longues
- Mise en place de BullMQ et du scheduler de publications differees

7.4 Phase 4 - Analytics et Gouvernance (semaines 11-14)

La quatrieme phase introduit les analytics avances et la gouvernance multi-approbateurs. Le dashboard est enrichi avec des graphiques detailles sur les performances de publication. Le workflow de validation evolue vers un systeme multi-etapes avec des approbateurs differents. Les notifications email et Slack sont activees pour les evenements cles du workflow.

- Ajout de l'API /api/analytics avec tous les endpoints
- Dashboard analytics avec KPI cards et graphiques
- Ajout du modele ValidationStep et du workflow multi-approbateurs
- Ecran de workflow de validation avec timeline
- Approbation en masse et file d'attente de validation
- Ajout du modele Notification et du service de notifications
- Integration email (Resend) et Slack Webhook
- Preferences de notification par utilisateur

7.5 Phase 5 - Multi-reseaux et Industrialisation (semaines 15-18)

La cinquieme et derniere phase prepare l'extension multi-reseaux et renforce l'industrialisation de la plateforme. Les architectures d'adaptateurs pour X/Twitter, Facebook, Instagram et Threads sont mises en place. Le monitoring, le logging structure et l'audit de securite sont renforces. Des tests de charge sont effectues et la documentation operationnelle est finalisee.

- Architecture d'adaptateurs multi-reseaux (interface SocialPublisher)
- Implementation de l'adaptateur X/Twitter
- Preparation des adaptateurs Facebook, Instagram, Threads (stubs)
- API de publication multi-reseaux
- Monitoring : health checks, metriques d'API, alertes
- Logging structure (pino) avec export vers un systeme de logs centralise
- Audit de securite : chiffrement des tokens, rotation des secrets
- Tests de charge et optimisation des performances
- Documentation operationnelle et runbooks

7.6 Script de migration automatise

Un script de migration automatise est fourni pour transformer les donnees V1 en donnees V2. Ce script gere la migration du schema (ajout des nouvelles tables et colonnes), la migration des donnees (creation du workspace par default, reconciliation des comptes LinkedIn) et la validation post-migration (verification de l'integrite des donnees). Le script est idempotent et peut etre relance en cas d'echec partiel.

```
// scripts/migrate-v1-to-v2.ts

async function migrateV1toV2() {
  console.log('Debut de la migration V1 vers V2...');

  // 1. Creer le workspace par default
  const defaultWorkspace = await prisma.workspace.create({
    data: {
      name: 'Espace par default',
      slug: 'default',
      isActive: true,
    }
  });

  // 2. Migrer les utilisateurs vers le workspace par default
  const users = await prisma.user.findMany();
  for (const user of users) {
    await prisma.workspaceMember.create({
      data: {
        workspaceId: defaultWorkspace.id,
        userId: user.id,
        role: user.role === 'admin' ? 'owner' : 'member',
      }
    });
  }

  // 3. Migrer les posts (ajouter workspaceId)
  await prisma.post.updateMany({
    data: { workspaceId: defaultWorkspace.id }
  });

  // 4. Migrer LinkedInAccount vers SocialAccount
  const accounts = await prisma.linkedInAccount.findMany();
  for (const acc of accounts) {
    await prisma.socialAccount.create({
      data: {
        workspaceId: defaultWorkspace.id,
        platform: 'linkedin',
        platformUserId: acc.organizationId ?? 'personal',
        displayName: acc.organizationName ?? 'Compte LinkedIn',
        accessToken: acc.accessToken,
        refreshToken: acc.refreshToken,
        tokenExpiresAt: acc.tokenExpiresAt,
        isActive: acc.isActive,
      }
    });
  }
}
```



```

// 5. Creer les evenements calendrier pour les posts planifies
const scheduledPosts = await prisma.post.findMany({
  where: { scheduledDate: { not: null } }
});
for (const post of scheduledPosts) {
  await prisma.calendarEvent.create({
    data: {
      workspaceId: defaultWorkspace.id,
      postId: post.id,
      title: post.subject,
      startDate: post.scheduledDate!,
      eventType: 'post',
    }
  });
}

// 6. Creer le profil de marque par default
await prisma.brandProfile.create({
  data: {
    workspaceId: defaultWorkspace.id,
    brandTone: 'professional',
    brandVoice: '',
  }
});

console.log('Migration V1 vers V2 terminee avec succes !');
}

```

8. Strategie produit V1 vers V2

8.1 Positionnement

La V1 est un MVP (Minimum Viable Product) qui a demontre la valeur de l'automatisation de la creation et publication de posts LinkedIn. La V2 transforme ce MVP en un produit complet de Content Operations, positionne comme un outil de gestion editorial professionnelle pour les equipes marketing B2B. Le produit se differencie par son approche IA-native (generation de contenu intelligente avec scoring), sa gouvernance avancee (workflows multi-approbateurs) et sa preparation multi-reseaux.

8.2 Roadmap produit

Phase	Period e	Livrables cles	KPIs
-------	-------------	----------------	------

Phase 1 : Fondations	S1-S3	PostgreSQL, Workspace, Middleware, Config multi-env	Zero regression V1, temps de reponse API < 200ms
Phase 2 : Branding	S4-S6	BrandProfile, PromptTemplate, Integration IA branding	Taux d'adoption du branding > 80%, score qualite moyen > 70
Phase 3 : Calendrier	S7-S10	Calendrier drag/drop, Media Library, Generation enrichie, Scheduler	Utilisation calendrier quotidienne, uploads medias > 50/semaine
Phase 4 : Analytics	S11-S14	Dashboard analytics, Workflow multi-etapes, Notifications	Taux de publication reussie > 95%, temps de validation < 24h
Phase 5 : Multi-reseaux	S15-S18	Adaptateurs sociaux, Monitoring, Audit securite	Couverture multi-reseaux, uptime > 99.5%

Tableau 26 - Roadmap produit V1 vers V2

8.3 Metriques de succes

Le succes de la V2 est mesure selon quatre axes : adoption utilisateur, qualite du contenu, efficacite operationnelle et fiabilite technique. Chaque axe dispose de KPIs specifiques qui sont suivis via le dashboard analytics et des outils de monitoring externes (Datadog, Sentry).

Axe	KPI	Cible
Adoption	Utilisateurs actifs hebdomadaires	Augmentation de 50% par rapport a la V1
Adoption	Taux d'utilisation du calendrier	80% des posts planifies via le calendrier
Qualite	Score moyen des posts publies	Superieur a 70/100
Qualite	Taux d'utilisation des templates de prompts	Superieur a 60%
Efficacite	Temps moyen de creation d'un post	Inferieur a 15 minutes
Efficacite	Temps moyen de validation	Inferieur a 24 heures
Fiabilite	Taux de publication reussie	Superieur a 95%
Fiabilite	Uptime de la plateforme	Superieur a 99.5%
Fiabilite	Temps de reponse API (p95)	Inferieur a 500ms

Tableau 27 - Metriques de succes de la V2

8.4 Model d'evolution et scalabilite

L'architecture V2 est concue pour supporter trois niveaux d'evolution future. Le premier niveau est l'enrichissement des fonctionnalites existantes (nouveaux types de contenu IA, nouveaux reseaux sociaux, nouvelles integrations de notification). Le deuxieme niveau est l'extraction en microservices si le trafic l'exige (service d'IA dedie, service de publication dedie, service d'analytics dedie). Le troisieme niveau est la transformation en plateforme multi-tenant avec SaaSification complete, facturation et gestion des

abonnements.

- **Court terme (6 mois)** : Enrichissement des features existantes, ajout de reseaux sociaux
- **Moyen terme (12 mois)** : Microservices, API publique, SDK partenaires
- **Long terme (18+ mois)** : Multi-tenant SaaS, marketplace de templates IA, analytics predictive

8.5 Risques et mitigation

Tout projet d'evolution majeur comporte des risques identifiés qui doivent etre anticipés et maîtrisés. Le tableau ci-dessous presente les principaux risques de la migration V1 vers V2 et les strategies de mitigation associees. Chaque risque est evalúe selon sa probabilité et son impact sur le projet.

Risque	Probabilité	Impact	Mitigation
Regression fonctionnelle V1	Moyenne	Critique	Tests de non-regression automatisés, deploiement progressif, rollback rapide
Migration de données incomplete	Faible	Critique	Script de migration idempotent, validation post-migration, backup pre-migration
Performance dégradée PostgreSQL	Moyenne	Moyen	Index optimisés, connection pooling (PgBouncer), requetes N+1 éliminées
Echec publications différées	Moyenne	Moyen	BullMQ avec retries, monitoring temps reel, alertes Slack
Resistance au changement	Élevée	Moyen	Formation utilisateurs, documentation, feedback continu, beta testeurs
Sécurité tokens OAuth	Faible	Critique	Chiffrement AES-256, rotation automatique, audit trimestriel

Tableau 28 - Risques et strategies de mitigation